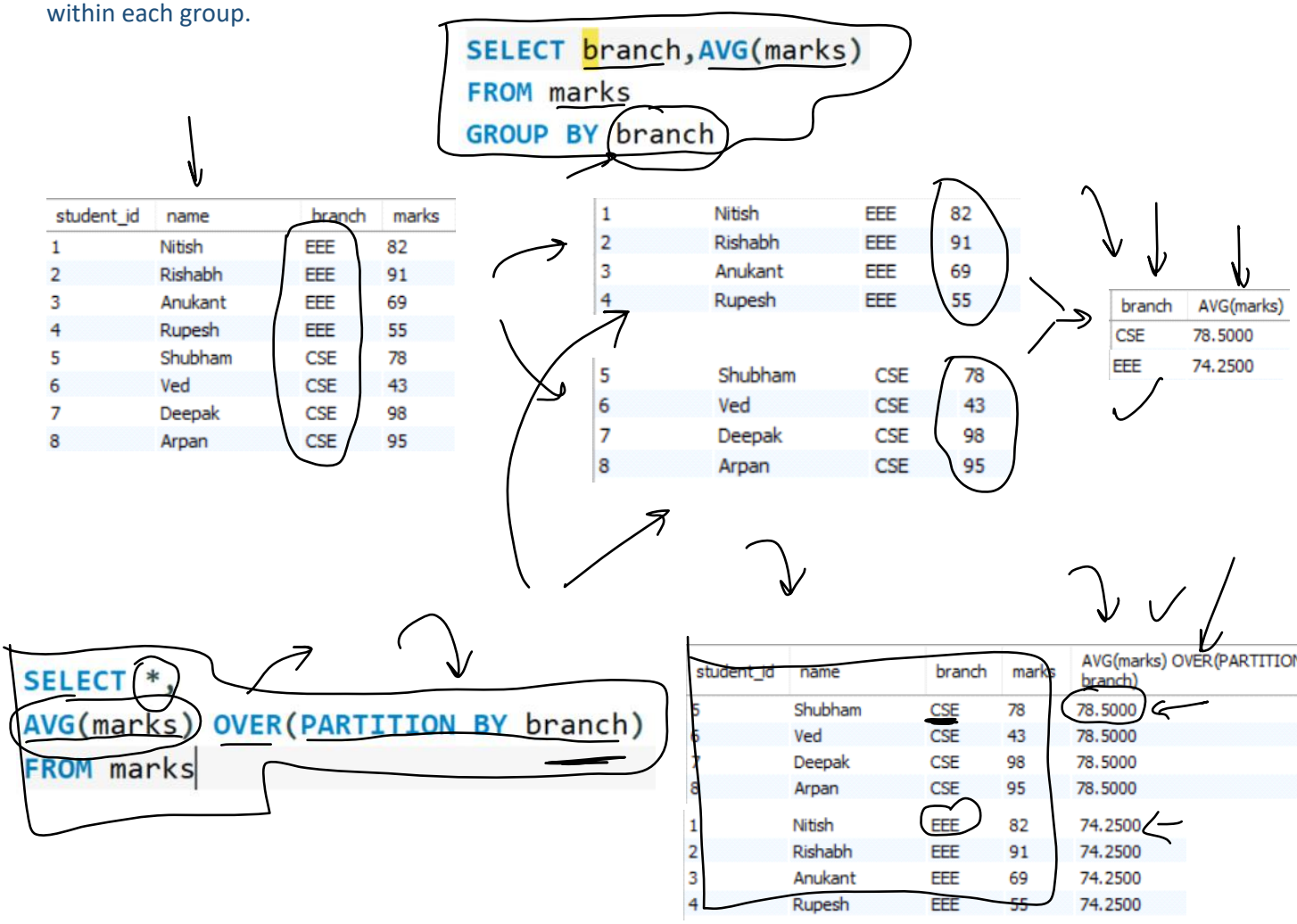
What are Window Functions?

27 February 2023 14:18

Window functions in SQL are a type of analytical function that perform calculations across a set of rows that are related to the current row, called a "window". A window function calculates a value for each row in the result set based on a subset of the rows that are defined by a window specification.

The window specification is defined using the OVER() clause in SQL, which specifies the partitioning and ordering of the rows that the window function will operate on. The partitioning divides the rows into groups based on a specific column or expression, while the ordering defines the order in which the rows are processed within each group.



Aggregate Function with OVER()

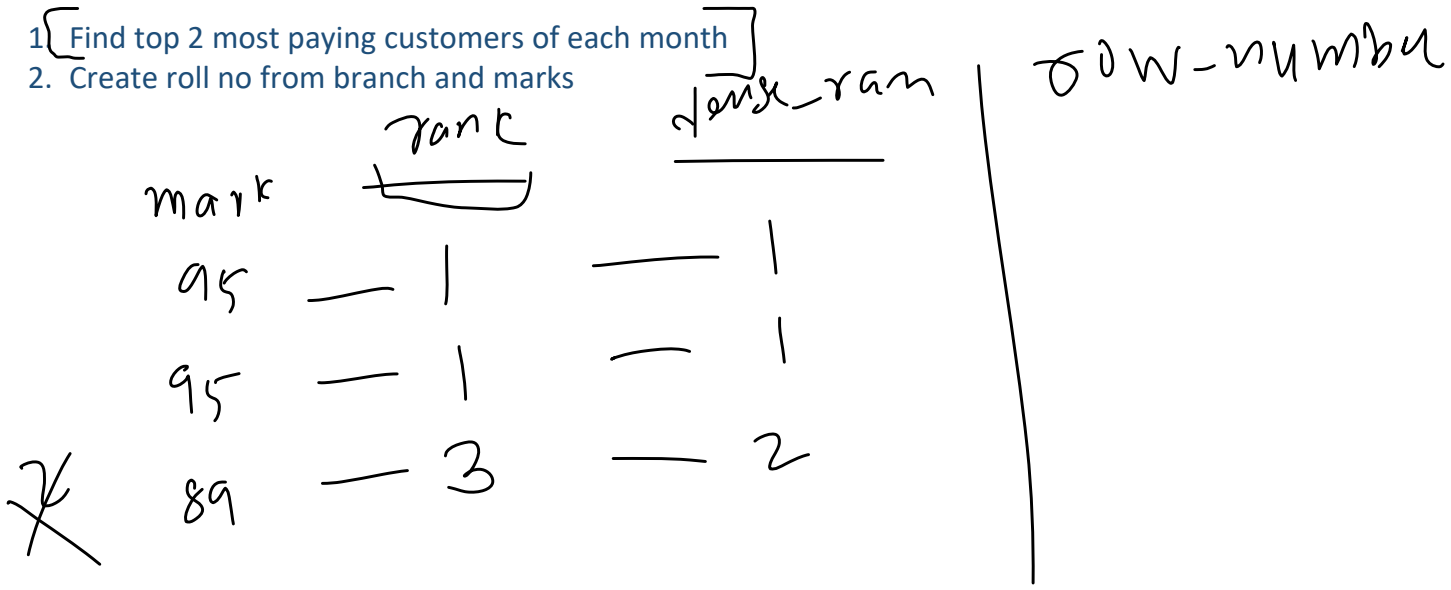
27 February 2023 16:41

Find all the students who have marks higher than the avg marks of their respective branch

RANK/DENSE_RANK/ROW_NUMBER

27 February 2023 16:56

1. Find top 2 most paying customers of each month
2. Create roll no from branch and marks



FIRST_VALUE/LAST VALUE/NTH_VALUE

27 February 2023 16:56

1. Find the branch toppers
2. FRAME Clause
3. Find the last guy of each branch
4. Alternate way of writing Window functions
5. Find the 2nd last guy of each branch, 5th topper of each branch

Frames

27 February 2023 19:08

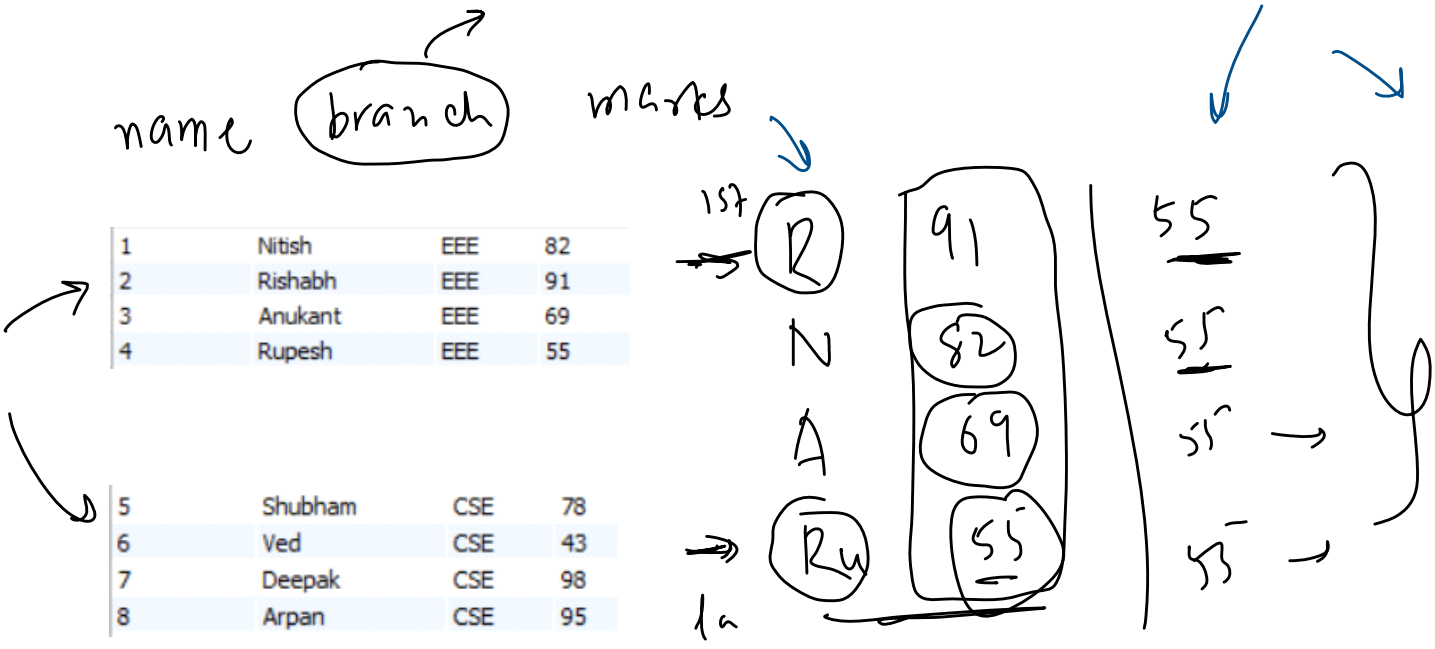
A frame in a window function is a subset of rows within the partition that determines the scope of the window function calculation. The frame is defined using a combination of two clauses in the window function: **ROWS** and **BETWEEN**.

The ROWS clause specifies how many rows should be included in the frame relative to the current row. For example, ROWS 3 PRECEDING means that the frame includes the current row and the three rows that precede it in the partition.

The BETWEEN clause specifies the boundaries of the frame.

Examples

- **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** means that the frame includes all rows from the beginning of the partition up to and including the current row.
- **ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING**: the frame includes the current row and the row immediately before and after it.
- **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**: the frame includes all rows in the partition.
- **ROWS BETWEEN 3 PRECEDING AND 2 FOLLOWING**: the frame includes the current row and the three rows before it and the two rows after it.



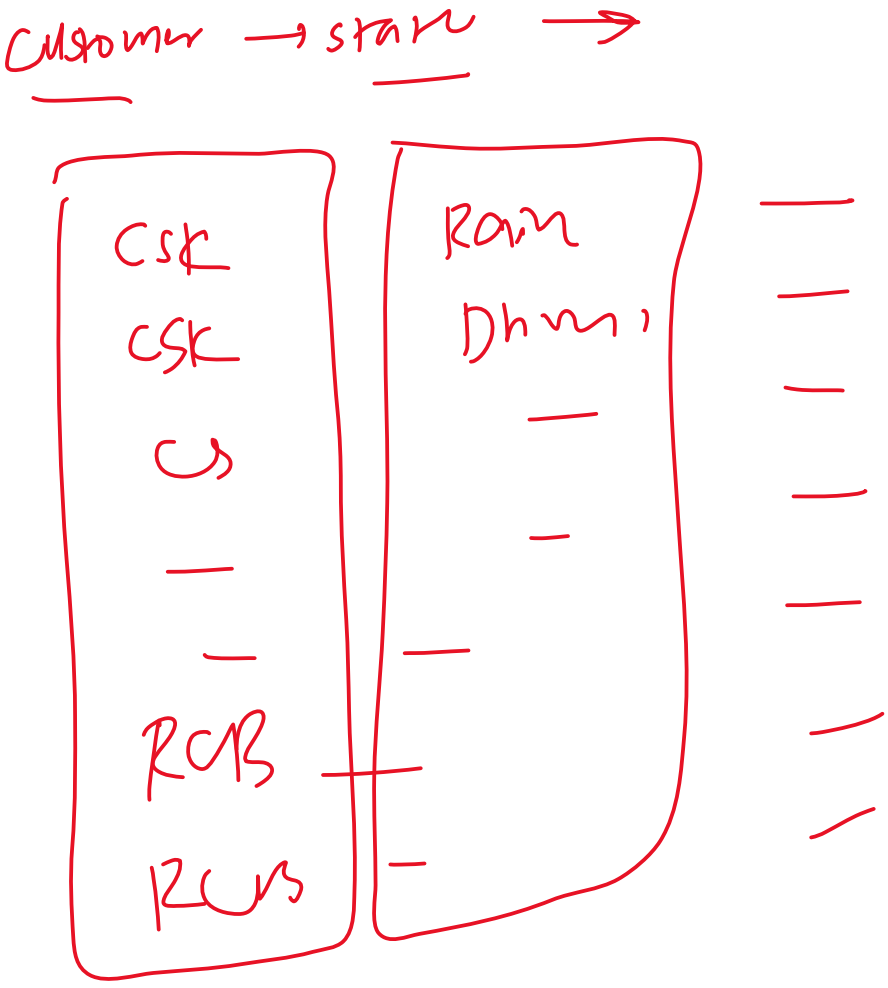
LEAD & LAG

27 February 2023 17:12

Find the MoM revenue growth of Zomato

Ranking

01 March 2023 13:57

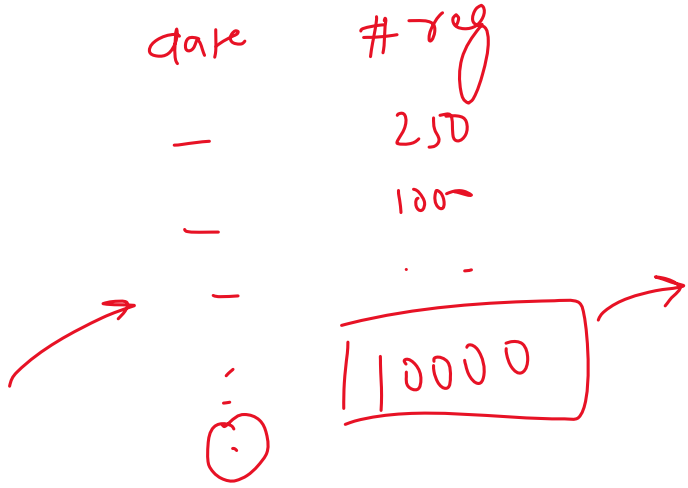


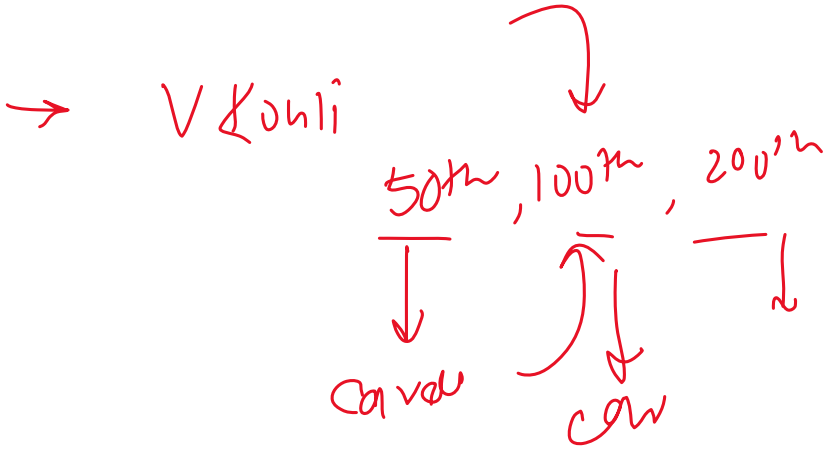
Cumulative Sum

01 March 2023 13:58

Cumulative sum is another type of calculation that can be performed using window functions. A cumulative sum calculates the sum of a set of values up to a given point in time, and includes all previous values in the calculation.

month	year	sales	cumulative_sum
1	2020	100	100
2	2020	150	250
3	2020	200	450
4	2020	175	625
5	2020	250	875
6	2020	300	1175
7	2020	275	1450
8	2020	200	1650





Cumulative Average

01 March 2023 13:58

Cumulative average is another type of average that can be calculated using window functions. A cumulative average calculates the average of a set of values up to a given point in time, and includes all previous values in the calculation.

Student_ID	Test_Number	Score	Cumulative_Avg
1	1	85	85.00
1	2	90	87.50
1	3	80	85.00
2	1	70	70.00
2	2	75	72.50
2	3	80	75.00
3	1	90	90.00
3	2	95	92.50
3	3	90	91.67



Running Average

01 March 2023 13:58

Running average (also known as moving average) is a statistical technique that calculates the average value of a dataset over a moving window of consecutive data points.

The window size determines the number of data points used to calculate the average, and as the window moves forward in time, the average is recalculated using the new data points and dropping the oldest one. This means that the running average is continuously updated and reflects the most recent trends in the data.

For example, a running average of a batsman's runs scored over a window of 10 matches will calculate the average runs scored in the last 10 matches, then move the window one match forward and recalculate the average for the new set of 10 matches, and so on.

Running averages are often used in finance, economics, and engineering to smooth out noisy or volatile data series, and to identify trends or patterns that may be obscured by random fluctuations in the data.

window = 5 matches

current form

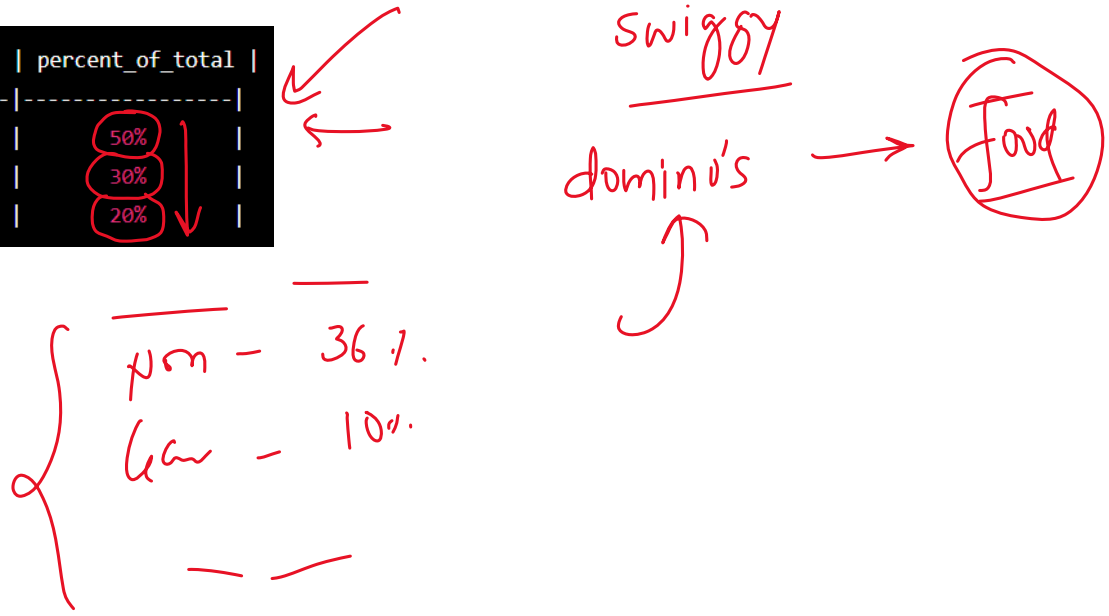
match_id	runs_scored	running_avg	cumulative_avg
1	52	52.0	52.0
2	41	46.5	46.5
3	17	36.7	36.7
4	68	44.5	44.5
5	36	42.8	42.8
6	91	49.2	50.0
7	22	44.0	45.1
8	55	44.9	45.6
9	81	51.2	48.9
10	13	41.6	45.6
11	29	41.5	45.3
12	44	42.3	45.2
13	36	41.4	44.8
14	72	47.9	45.8
15	87	56.0	48.7

Percent of total

01 March 2023 13:59

Percent of total refers to the percentage or proportion of a specific value in relation to the total value. It is a commonly used metric to represent the relative importance or contribution of a particular value within a larger group or population.

category	total_sales	percent_of_total
Category A	500	50%
Category B	300	30%
Category C	200	20%

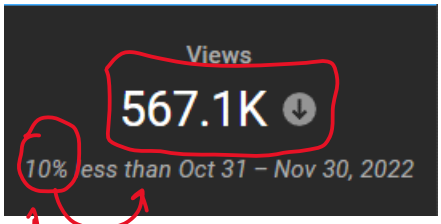


Percent Change

01 March 2023 13:59

Percent change is a way of expressing the difference between two values as a percentage of the original value. It is often used to measure how much a value has increased or decreased over a given period of time, or to compare two different values.

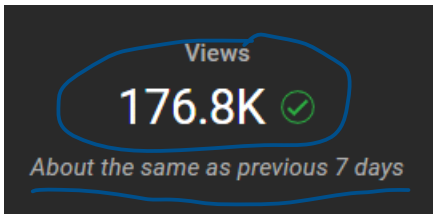
percent change = ((new value - old value) / old value) x 100



Dec



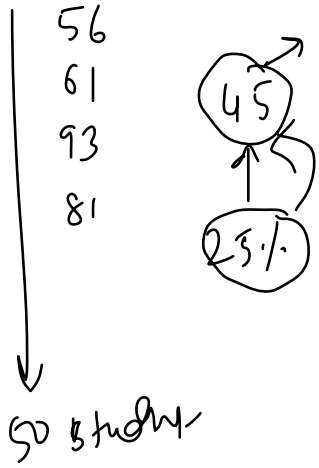
Jan



Percentiles & Quantiles

01 March 2023 13:59

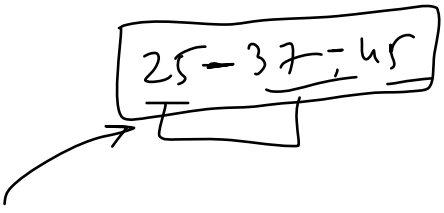
A **Quantile** is a measure of the distribution of a dataset that divides the data into any number of equally sized intervals. For example, a dataset could be divided into **deciles** (ten equal parts), **quartiles** (four equal parts), **percentiles** (100 equal parts), or any other number of intervals.



Each quantile represents a value below which a certain percentage of the data falls. For example, the 25th percentile (also known as the first quartile, or Q1) represents the value below which 25% of the data falls. The 50th percentile (also known as the median) represents the value below which 50% of the data falls, and so on.

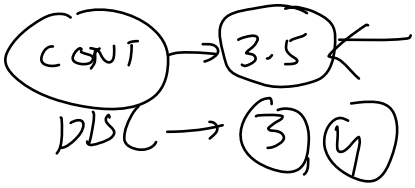
Q1. Find the median marks of all the students
Q2. Find branch wise median of student marks.





PERCENTILE_CONT calculates the continuous percentile value, which returns the interpolated value between adjacent data points. In other words, it estimates the percentile value by assuming that the values between data points are distributed uniformly. This function returns a value that may not be present in the original dataset.
PERCENTILE_DISC, on the other hand, calculates the discrete percentile value, which returns the value of the nearest data point. This function returns a value that is present in the original dataset.

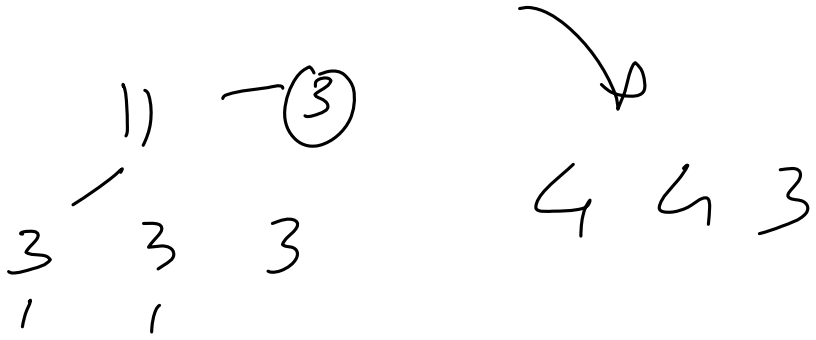
For example if we have 1,2,3,4,4,5



Segmentation

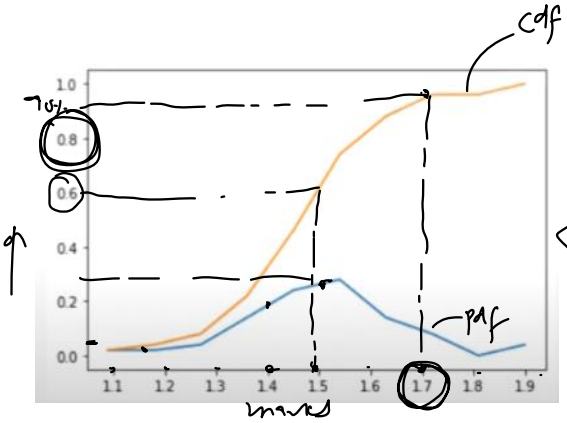
02 March 2023 09:30

Segmentation using NTILE is a technique in SQL for dividing a dataset into equal-sized groups based on some criteria or conditions, and then performing calculations or analysis on each group separately using window functions.



Cumulative Distribution

02 March 2023 09:03



The cumulative distribution function is used to describe the probability distribution of random variables. It can be used to describe the probability for a discrete, continuous or mixed variable. It is obtained by summing up the probability density function and getting the cumulative probability for a random variable

"What percentage of the rows in the data set have a value less than or equal to the current row?"

CUME_DIST = (number of rows with a value less than or equal to the current row) / (total number of rows)

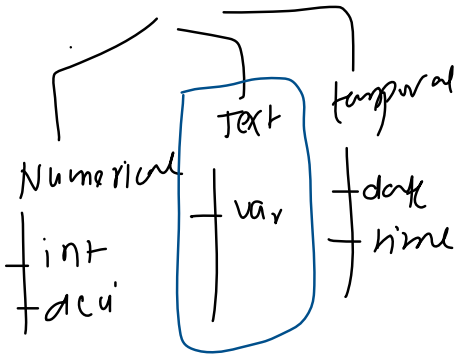
Partition By multiple columns

01 March 2023 14:00

String Data Types

03 March 2023 14:44

1. **CHAR**: This data type is used to store fixed-length strings. The length of the string is specified when the table is created, and the field will always use that amount of space, regardless of whether the string stored in it is shorter or longer. For example, if you define a CHAR(10) field and store the string "hello" in it, MySQL will pad the string with spaces so that it takes up 10 characters. CHAR fields are useful when you have a field that always contains the same length of data, such as a state abbreviation or a phone number.
2. **VARCHAR**: This data type is used to store variable-length strings. The length of the string can be up to a specified maximum, but the field will only use as much space as it needs to store the actual data. For example, if you define a VARCHAR(10) field and store the string "hello" in it, MySQL will only use 5 characters to store the data. VARCHAR fields are useful when you have a field that can contain varying amounts of data, such as a user's name or address.
3. **TEXT**: This data type is used to store larger amounts of variable-length string data than VARCHAR. It can store up to 65,535 characters. TEXT fields are useful when you need to store large amounts of text data, such as blog posts or comments.
4. **MEDIUMTEXT**: This data type is used to store even larger amounts of text data than TEXT. It can store up to 16,777,215 characters. MEDIUMTEXT fields are useful when you need to store very large amounts of text data, such as long-form articles or legal documents.
5. **LONGTEXT**: This data type is used to store the largest amounts of text data. It can store up to 4,294,967,295 characters. LONGTEXT fields are useful when you need to store extremely large amounts of text data, such as entire books or large collections of data.



char(10)

hello

Text

Wildcards

03 March 2023 14:47

The LIKE operator in MySQL is used to match a string value against a pattern using wildcard characters. It is commonly used in SELECT, WHERE, and JOIN clauses to filter or join rows based on a pattern match.

The LIKE operator uses two wildcard characters: the **percent sign (%)** and the **underscore (_)**. The percent sign represents zero, one, or more characters, while the underscore represents a single character.



String Functions

03 March 2023 14:51

- **upper/lower**
- **concat & concat_ws**
- **substr -> last 5 chars**
- **replace**
- **reverse -> palindrome**
- **char_length vs length** -> where both are not same -
  The main difference between CHAR_LENGTH and LENGTH is that CHAR_LENGTH returns the length of a string in characters, while LENGTH returns the length of a string in bytes. This difference is important when dealing with multi-byte character sets, such as UTF-8, where a single character may be represented by multiple bytes. Example - café

- **insert**(str, pos, len newstr)
  - str: The original string to insert into.
  - pos: The position at which to insert the new substring. The first position is 1.
  - len: The number of characters to replace.
  - newstr: The new substring to insert.

- **left and right**
- **repeat**
- **trim[ltrim and rtrim]**
- **substring_index(Split)** - www.campusx.in
- **strcmp** -
  The STRCMP() function returns an integer that indicates the relationship between the two strings:
  - If str1 is less than str2, the function returns a negative integer.
  - If str1 is greater than str2, the function returns a positive integer.
  - If str1 is equal to str2, the function returns 0.

- **locate("world", "hello world")**
- **lpad and rpad** ('hello', 10, '*')

Data Cleaning

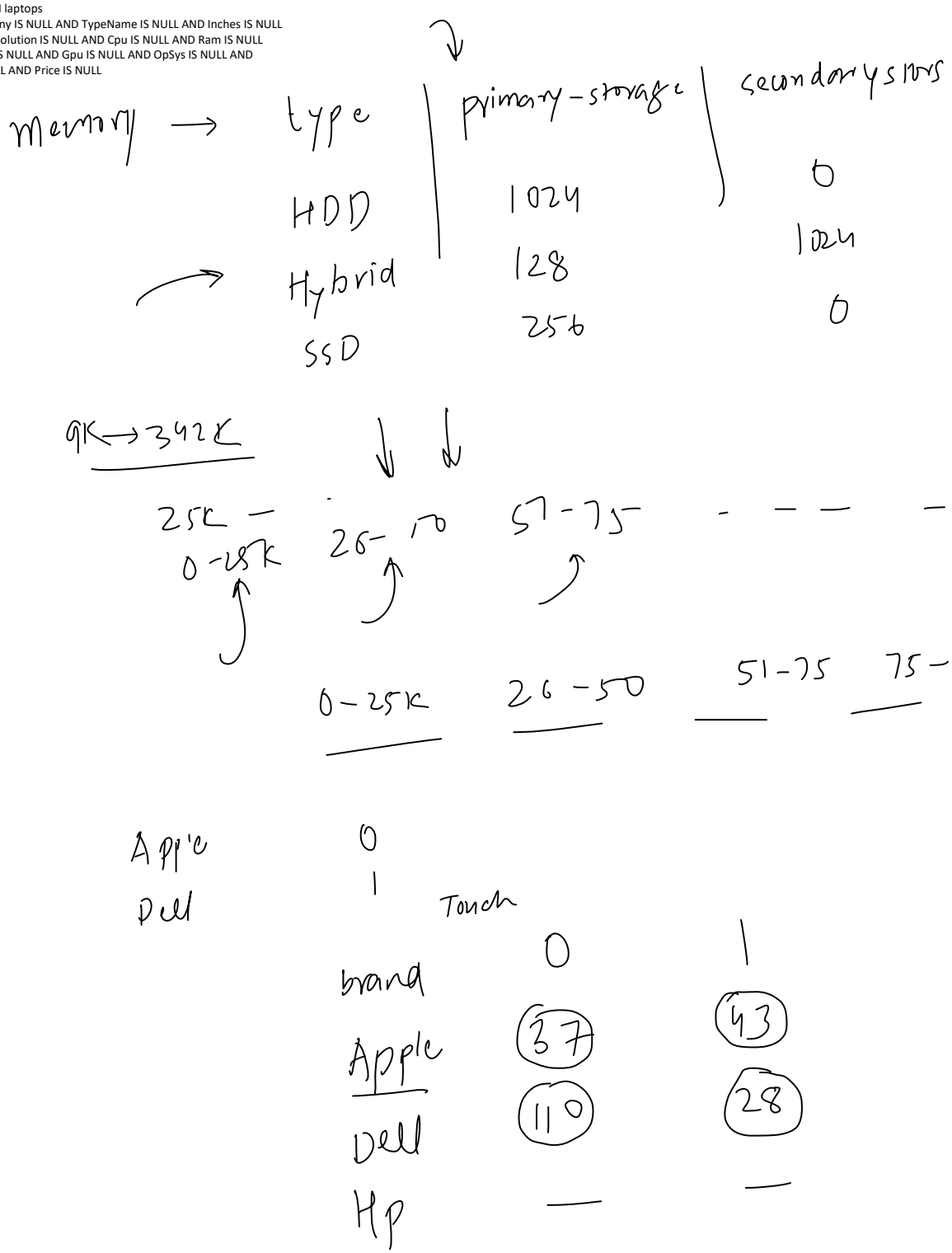
03 March 2023 17:03

1. Create backup
2. Check number of rows
3. Check memory consumption for reference
4. Drop non important cols
5. Drop null values
6. Drop duplicates
7. Clean RAM -> change col data type
8. Clean weight -> change col type
9. ROUND price col and change to integer
10. Change the OpSys col
11. Gpu
12. Cpu

Extra

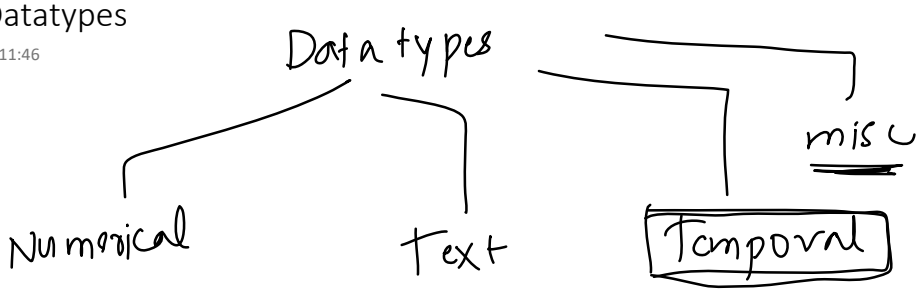
03 March 2023 17:14

```
SELECT * FROM laptops
WHERE Company IS NULL AND TypeName IS NULL AND Inches IS NULL
AND ScreenResolution IS NULL AND Cpu IS NULL AND Ram IS NULL
AND Memory IS NULL AND Gpu IS NULL AND OpSys IS NULL AND
WEIGHT IS NULL AND Price IS NULL
```



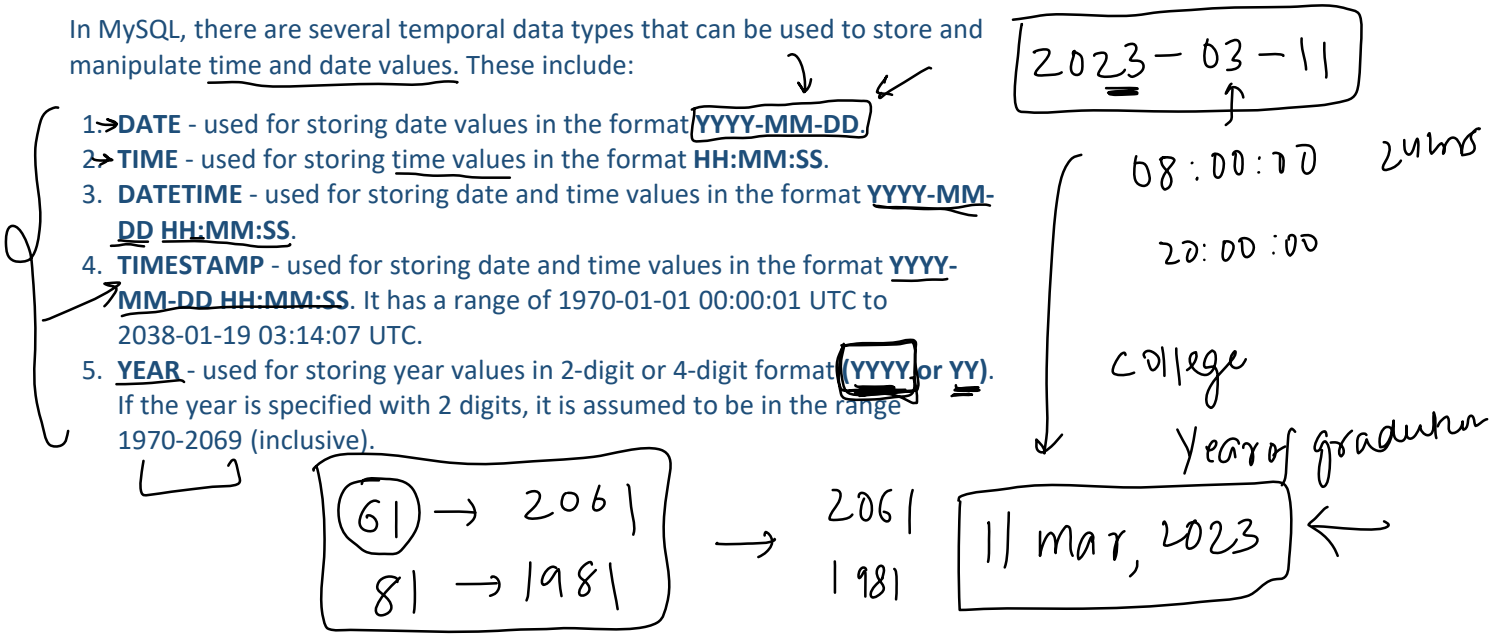
Temporal Datatypes

24 February 2023 11:46



In MySQL, there are several temporal data types that can be used to store and manipulate time and date values. These include:

1. **DATE** - used for storing date values in the format **YYYY-MM-DD**.
2. **TIME** - used for storing time values in the format **HH:MM:SS**.
3. **DATETIME** - used for storing date and time values in the format **YYYY-MM-DD HH:MM:SS**.
4. **TIMESTAMP** - used for storing date and time values in the format **YYYY-MM-DD HH:MM:SS**. It has a range of 1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC.
5. **YEAR** - used for storing year values in 2-digit or 4-digit format (**YYYY or YY**). If the year is specified with 2 digits, it is assumed to be in the range 1970-2069 (inclusive).



Creating and Populating Temporal Tables

24 February 2023 11:47

1. Uber -> user_id, cab_id, start_time, end_time

DATETIME Functions

24 February 2023 11:50

1. CURR_DATE()
2. CURR_TIME()
3. NOW()

Extraction Function

1. DATE() and TIME()
2. YEAR()
3. DAY() or DAYOFMONTH()
4. DAYOFWEEK()
5. DAYOFYEAR()
6. MONTH() and MONTHNAME()
7. QUARTER()
8. WEEK() or WEEKOFYEAR()
9. HOUR() -> MINUTE() -> SECOND()
10. LAST_DAY()

Datetime Formatting

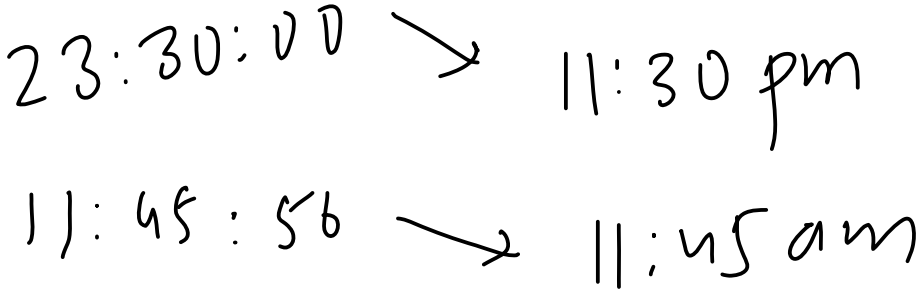
24 February 2023 11:48

DATE_FORMAT()
TIME_FORMAT()

Type conversion

24 February 2023 11:48

1. Implicit Type Conversion
2. Explicit Type Conversion -> STR_TO_DATE()



DATETIME Arithmetic

24 February 2023 11:50

1. DATEDIFF()
2. TIMEDIFF()
3. DATE_ADD() and DATE_SUB() INTERVAL
4. ADDTIME() and SUBTIME()

TIMESTAMP VS DATETIME

11 March 2023 11:10

In MySQL, both DATETIME and TIMESTAMP are used to store date and time values, but they differ in their range, storage format, and behaviour.

Here are the main differences between DATETIME and TIMESTAMP:

1. **Range**: DATETIME supports a range of '1000-01-01 00:00:00' to '9999-12-31 23:59:59', while TIMESTAMP supports a range of '1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' UTC.
2. **Storage format**: DATETIME uses 8 bytes to store the date and time values, while TIMESTAMP uses 4 bytes.
3. **Behaviour on insertion/update**: DATETIME values are stored as-is, without any conversion, while TIMESTAMP values are converted from the current time zone to UTC when inserted, and converted back to the current time zone when retrieved.
4. **Precision**: DATETIME can store up to microseconds (6 digits after the decimal point), while TIMESTAMP can only store up to seconds.
5. **Auto-update**: TIMESTAMP columns can be set to update automatically whenever the row is inserted or updated, using the ON UPDATE CURRENT_TIMESTAMP clause.

In general, you should use DATETIME when you need to store date and time values outside the range of TIMESTAMP, or when you need to store values with greater precision than TIMESTAMP. You should use TIMESTAMP when you need to store values that can be automatically updated, or when you want to take advantage of its smaller storage format.

Views

24 March 2023 17:37

- What are Views?

In SQL, a view is a virtual table that does not store any data on its own but presents a customized view of one or more tables in a database. A view can be thought of as a pre-defined SELECT statement that retrieves data from one or more tables and returns a specific subset of data to the user.

So basically it is a logical table instead of a physical table

Once a view is created, it can be used in the same way as a table in SQL queries, and any changes made to the underlying tables will be reflected in the view.(Show)

Simple Views - Created from 1 single table

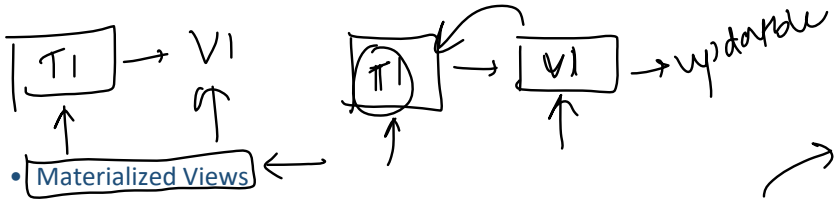
Complex Views - Created from multiple tables with the help of joins, subquery etc.

- Read only Vs Updatable Views

a. **Read-only views**: As the name suggests, read-only views are views that cannot be updated. They are used to simplify the process of querying data, but they cannot be used to modify or delete data in the underlying tables.

b. **Updatable views**: Updatable views are views that allow you to modify, insert or delete data in the underlying tables through the view. They behave like normal tables, but with restrictions.

To make a view updatable, certain conditions must be met. For example, the view must not contain any derived columns, subqueries, or aggregate functions. Additionally, the view must be based on a single table or a join of tables with a unique one-to-one relationship.

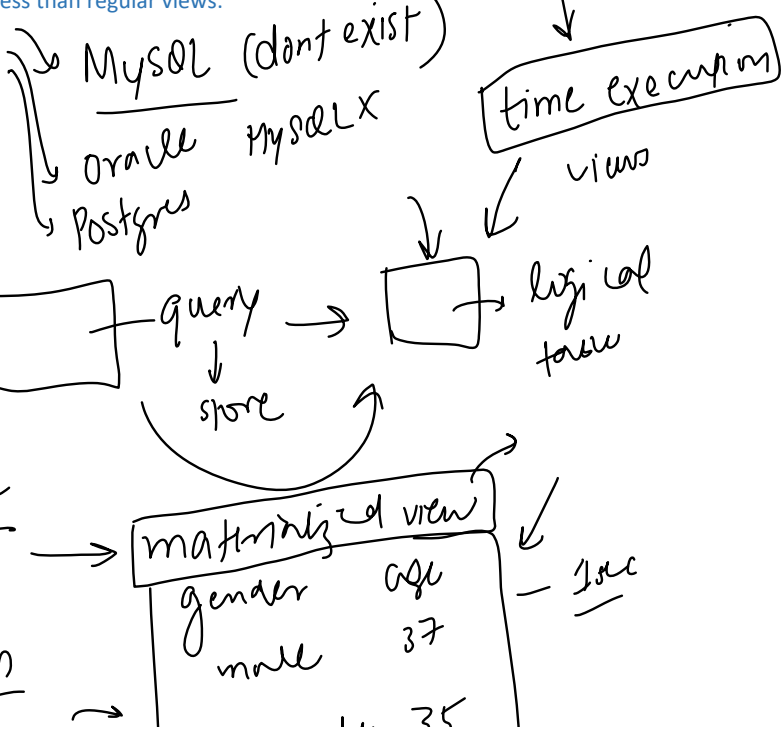


- Materialized Views

A materialized view is a database object in SQL that contains the results of a query. Unlike regular views, which are just virtual tables that store SQL queries, materialized views are physical tables that store the results of a query. Materialized views are precomputed and stored on disk, which makes them much faster to access than regular views.

Benefit - Faster queries

Disadvantage - Need to manually update the view

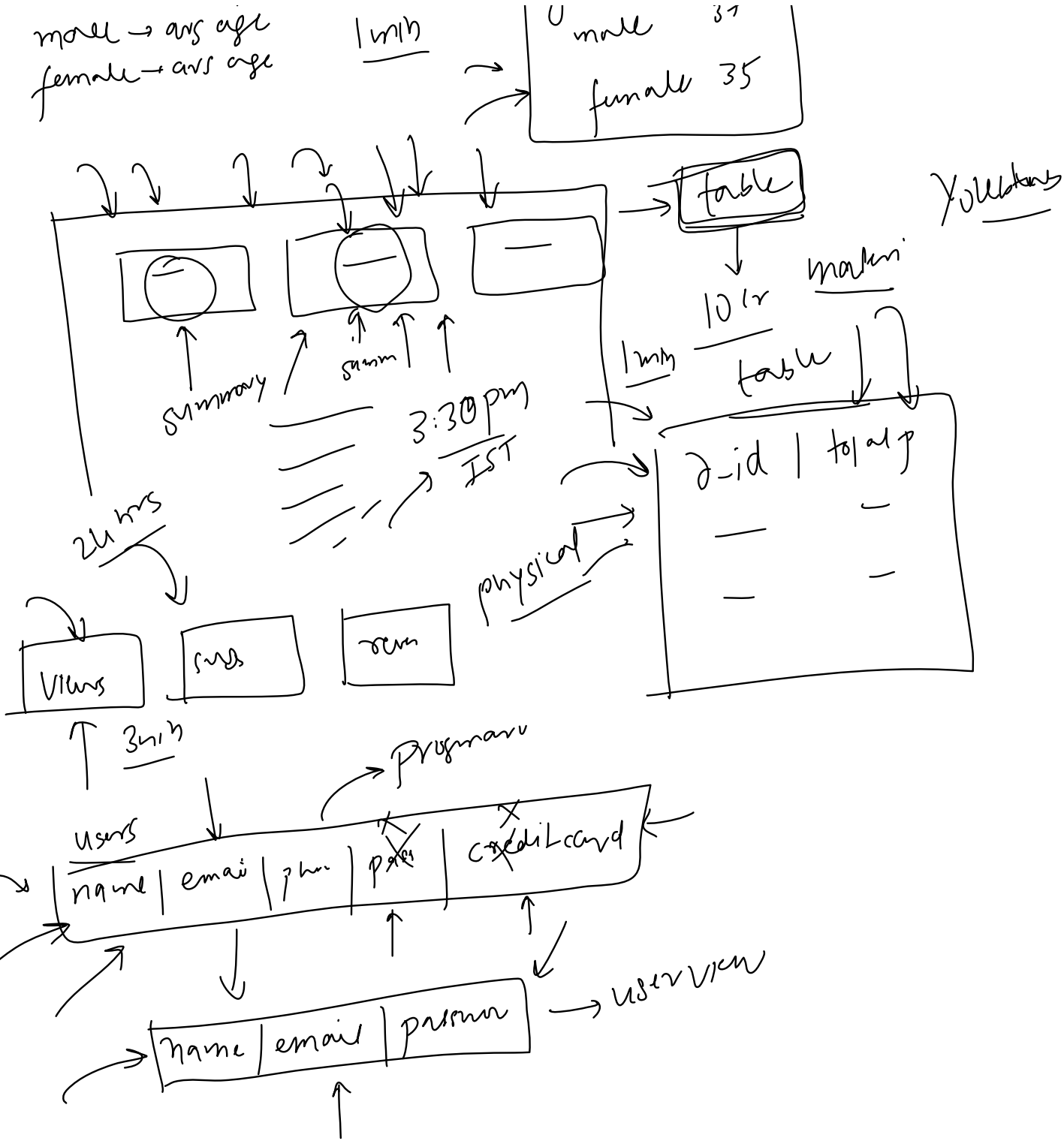


- Advantages of using a View

No Physical Storage

Security

Make complex queries simple

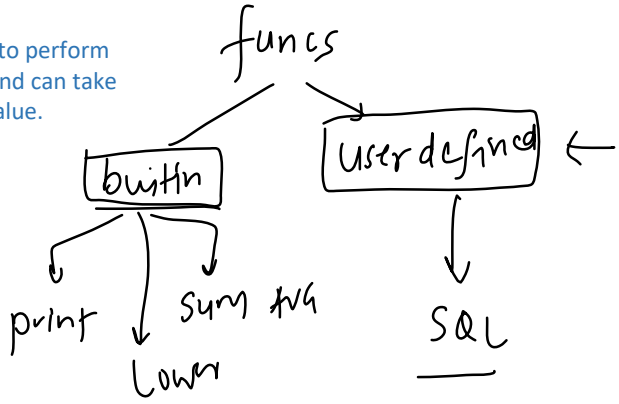

male → avg age
female → avg age
1 min
male
female 35
table
summary
3:30 pm
IST
24 hrs
1 min
10 hr
table
physical
Views
3 min
primary
users
name
email
password
credit card
name
email
password
user view

User Defined Functions

24 March 2023 18:58

- What are function and their advantages

User-defined functions (UDFs) in SQL are functions that are created by users to perform specific tasks. These functions can be used just like built-in functions in SQL and can take parameters as input, perform some operations on them, and then return a value.

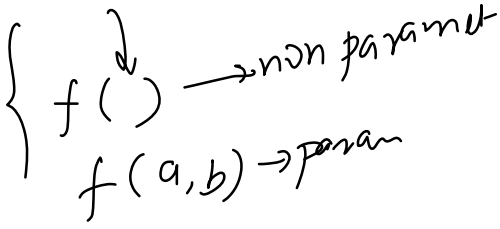


- Syntax

```
DELIMITER $$

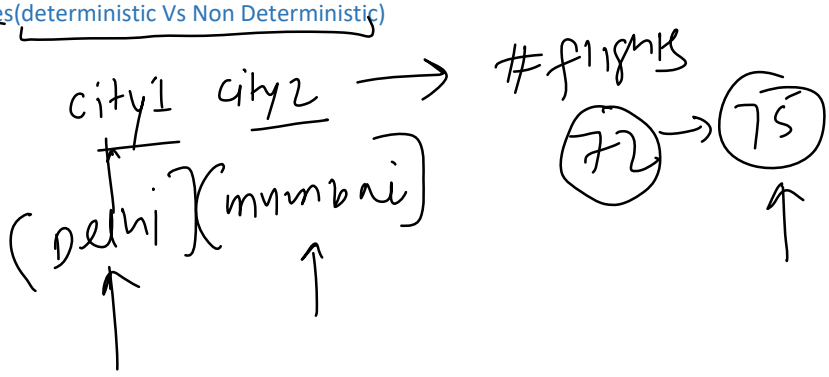
CREATE FUNCTION Function_Name(
    Parameter_1 DataType,
    Parameter_2 DataType,
    Parameter_n DataType,
)
RETURNS Return_Datatype
[NOT] DETERMINISTIC
BEGIN
      Function Body
      Return Return_Value
END $$

DELIMITER ;
```



Examples

- hello world
- parameterized vs non parameterized
- Calculate age in years(for col)
- greet with name -> conditional title
- Date formatting and flights between 2 cities(deterministic Vs Non Deterministic)
- show all functions of a database
- Drop function



Benefits

1. Simplifies SQL queries
2. Reusability
3. Enhances readability

debugging

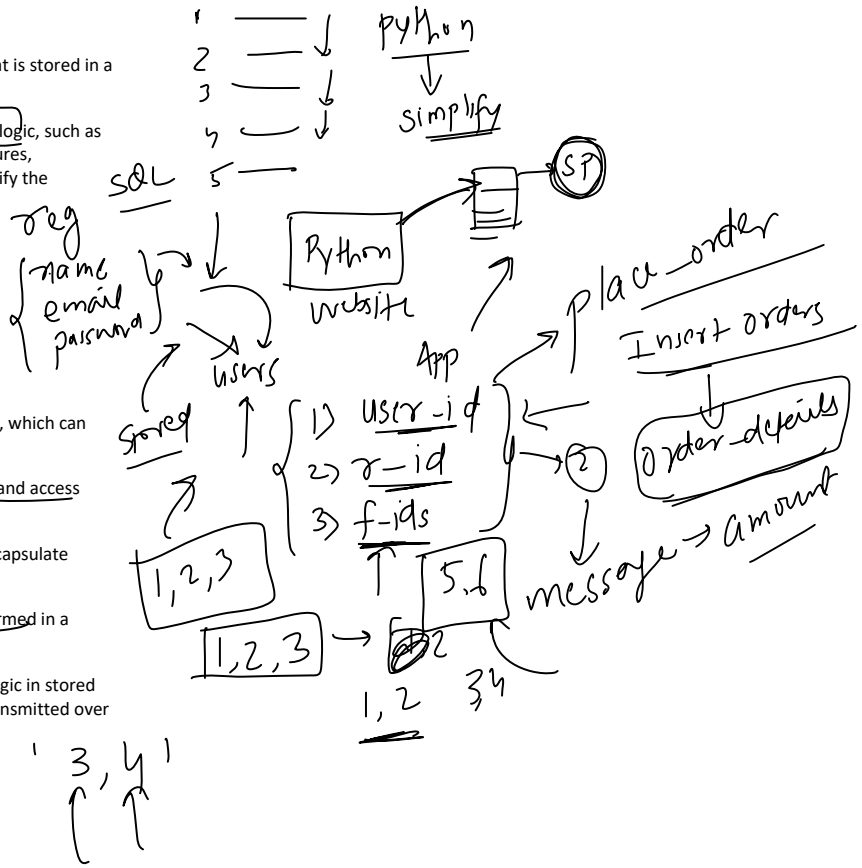
Stored Procedures

25 March 2023 16:39

A stored procedure is a named block of SQL statements and procedural logic that is stored in a database and can be executed by a user or application.

Stored procedures are often used to encapsulate business logic and application logic, such as data validation, data processing, and database updates. By using stored procedures, developers can separate application logic from the presentation layer and simplify the application code.

- Create hello world stored procedure
- Create stored procedure to create a new user
- Show error message
- Create stored procedure to show orders placed by 1 single user
- Create a stored procedure to place an order



Some of the benefits of using stored procedures include:

1. **Improved performance**: Stored procedures are precompiled and optimized, which can improve performance and reduce network traffic.
2. **Enhanced security**: Stored procedures can be granted specific permissions and access rights, which can improve security and limit access to sensitive data.
3. **Encapsulation of business logic**: Stored procedures allow developers to encapsulate complex business logic and make it easier to maintain and update.
4. **Consistency**: Stored procedures ensure that database operations are performed in a consistent manner, which can help to maintain data integrity.
5. **Reduced network traffic**: By encapsulating data access and manipulation logic in stored procedures, developers can reduce the amount of data that needs to be transmitted over the network.



Transactions

25 March 2023 08:19

What are Transactions?

A database transaction is a sequence of operations that are performed as a single logical unit of work in a database management system (DBMS). A transaction may consist of one or more database operations, such as inserts, updates, or deletes, which are treated as a single atomic operation by the DBMS.

It follows the principle of all or none.

What is Commit, Rollback and Savepoint?

In a database transaction, there are three main commands that are used to manage the transaction:

1. **Commit**: A commit command is used to permanently save the changes made by a transaction to the database. When a transaction is committed, all changes made by the transaction are made permanent and cannot be rolled back.
2. **Rollback**: A rollback command is used to undo the changes made by a transaction and return the database to its state before the transaction began. When a transaction is rolled back, all changes made by the transaction are discarded and the database is returned to its previous state.
3. **Savepoint**: A savepoint command is used to mark a specific point within a transaction where a rollback can be performed. This allows for partial rollbacks of a transaction, where only changes made after the savepoint are undone, while changes made before the savepoint are still committed to the database.

What is Autocommit?

Autocommit is a feature of database management systems (DBMS) that automatically commits each individual database transaction as soon as it is completed, rather than requiring an explicit commit command to be issued.

When Autocommit is enabled, each individual SQL statement issued against the database is treated as a separate transaction and is committed immediately after it is executed. This means that each SQL statement becomes a separate, independent transaction, and its effects are immediately visible to other users.

- Each SQL write statement is Autocommit (prove)
- set Autocommit = 0 and (show)
- START TRANSACTION -> show operations without committing
- START TRANSACTION -> with commit
- START TRANSACTION -> all or none with commit
- rollback
- rollback with savepoint
- rollback and commit together

What is ACID properties of a Transaction?

ACID is an acronym that stands for Atomicity, Consistency, Isolation, and Durability, which are a set of properties that ensure reliable database transactions:

1. **Atomicity**: This property ensures that a transaction is treated as a single, indivisible unit of work. This means that either all of the changes made by a transaction are committed to the database, or none of them are. If any part of the transaction fails, the entire transaction is rolled back, and all changes are undone.
2. **Consistency**: This property ensures that a transaction takes the database from one valid state to another valid state. It requires that all data in the database must conform to a set of rules, or constraints, which ensure data integrity.
3. **Isolation**: This property ensures that concurrent transactions do not interfere with each other. It requires that each transaction executes as if it were the only transaction executing against the database, even if multiple transactions are executing at the same time.
4. **Durability**: This property ensures that once a transaction is committed, its changes are permanently stored in the database, even in the event of a system failure or power outage. This is typically achieved through the use of database backups, replication, or other forms of data redundancy.

Together, these properties ensure that database transactions are reliable, consistent, and accurate, and that the data stored in a database is both protected and available at all times. The ACID properties are essential for mission-critical applications that require high levels of data integrity and availability, such as banking, finance, and healthcare systems.

